

Proyecto de Software

ORM: trabajar con objetos, comprender la base

De una solicitud en Python a una operación persistente.

Recorrido de la clase

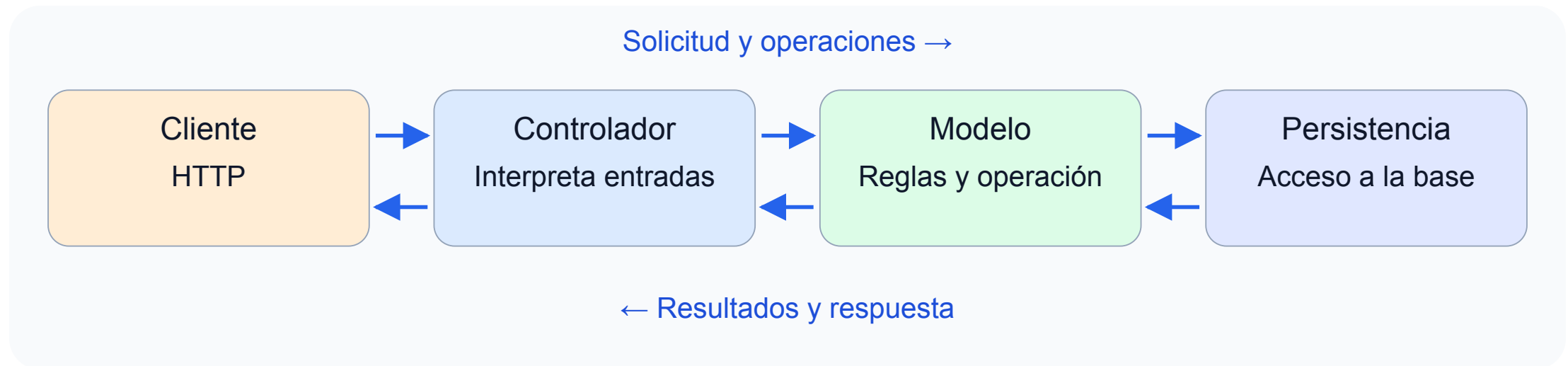
1. Qué cambia al incorporar un ORM.
2. Modelos e integración con Flask.
3. Consultas y relaciones.
4. Escrituras, sesión y transacciones.
5. Diagnóstico y revisión de código.

No buscamos memorizar una biblioteca: buscamos explicar lo que hace.

Recuperamos dos ideas

BBDD: los datos tienen restricciones y las operaciones necesitan garantías.

MVC: el controlador interpreta HTTP; el modelo concentra datos y reglas.

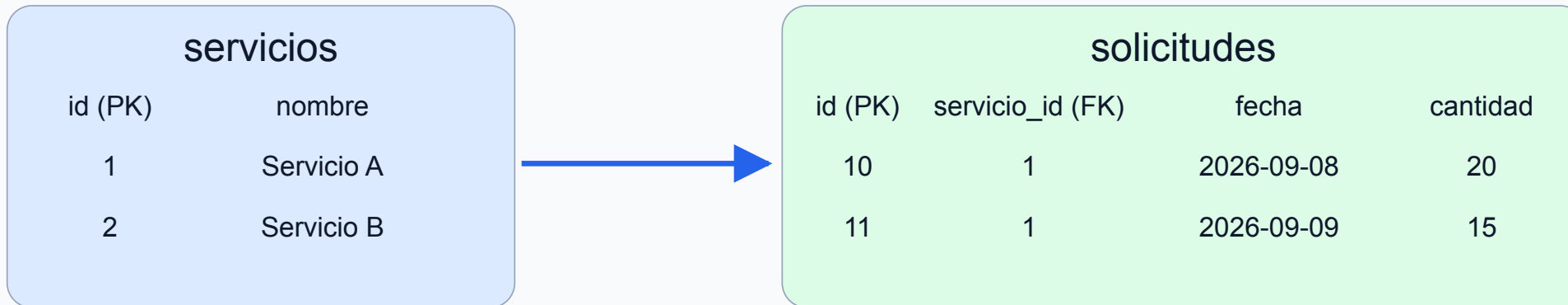


Hoy cambiamos parte del acceso a datos, no las responsabilidades.

Servicios y solicitudes

Un **servicio** puede tener muchas **solicitudes**.

Cada solicitud indica servicio, fecha y cantidad pedida.



PK: identifica una fila. FK: referencia una fila de otra tabla.

¿Qué trabajo repetimos?

```
# Con SQL explícito: recuperar filas y traducir sus datos.  
cur.execute(  
    "SELECT id, cantidad FROM solicitudes WHERE servicio_id = %s",  
    (servicio_id, ),  
)  
filas = cur.fetchall()
```

- Escribir operaciones parecidas sobre las mismas entidades.
- Traducir filas a estructuras Python y viceversa.
- Mantener esa traducción cuando cambia el modelo.

El SQL explícito sigue siendo una herramienta válida.

ORM: mapeo objeto-relacional

Un ORM relaciona objetos de la aplicación con datos de una base relacional y genera SQL para expresar parte de las operaciones.

En Python	En la base relacional
Clase mapeada	Tabla
Objeto persistente	Fila identificada por una clave
Atributo mapeado	Columna
Referencia entre objetos	Relación apoyada en claves

Esta correspondencia orienta el mapeo; no es una equivalencia universal.

La base sigue siendo relacional. El SQL sigue existiendo.

Qué resuelve y qué no

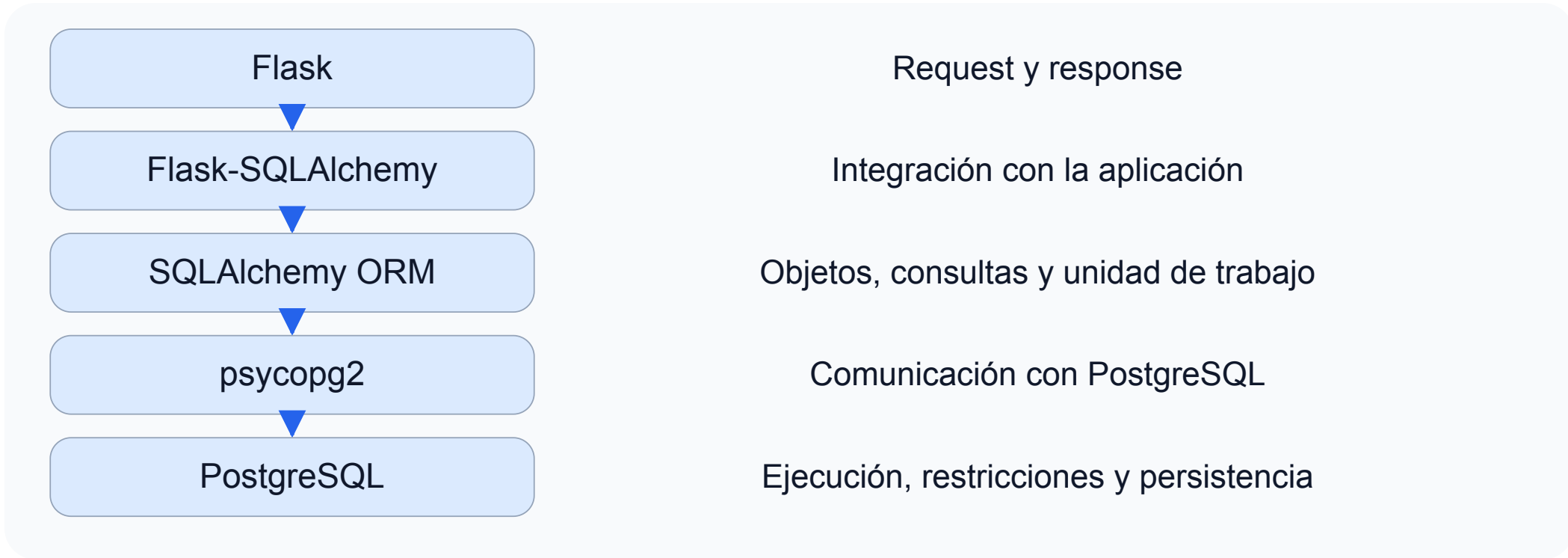
Ayuda a: expresar consultas, recuperar objetos y coordinar cambios.

No decide por nosotros:

- qué datos son válidos y quién puede modificarlos;
- dónde empieza y termina una operación;
- qué consultas resultan costosas;
- cómo conservar la historia del negocio.

Cambiar de motor tampoco es automático: hay diferencias de tipos y SQL.

Las piezas del recorrido



Flask-SQLAlchemy integra el ORM con el ciclo de vida de la aplicación.

SQLAlchemy y Flask-SQLAlchemy

SQLAlchemy	Flask-SQLAlchemy
Biblioteca independiente de Flask	Extensión para aplicaciones Flask
Proporciona ORM, consultas, motor y sesión	Configura e integra esos componentes
No conoce el ciclo de vida de Flask	Vincula la sesión al contexto de la aplicación

Flask-SQLAlchemy no es otro ORM: utiliza SQLAlchemy y simplifica su integración con Flask.

Dependencias del proyecto

```
poetry add 'SQLAlchemy>=2.0,<2.1' 'Flask-SQLAlchemy>=3.1,<3.2'  
poetry add psycopg2-binary python-dotenv
```

El proyecto utiliza Flask, Poetry y PostgreSQL.

- Una base PostgreSQL.
- Dependencias resueltas y registradas en `poetry.lock`.

Los módulos comparten la configuración y el objeto `db`.

Una extensión sin aplicación global

```
# src/core/db.py
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

db = SQLAlchemy(model_class=Base)
```

Creamos `db` una vez. La factory lo vincula con cada aplicación.

Configuración antes de inicialización

```
# src/web/__init__.py
import os
from dotenv import load_dotenv
from flask import Flask
from src.core.db import db

def create_app():
    load_dotenv()
    app = Flask(__name__)
    app.config["SQLALCHEMY_DATABASE_URI"] = os.environ["DATABASE_URL"]
    db.init_app(app)
    from src.core import models # registra las clases mapeadas
    # Aquí se registran los Blueprints de la aplicación.
    return app
```

.env :

```
DATABASE_URL=postgresql+psycopg2://usuario:clave@localhost/proyecto_db
```

Modelo: un servicio

```
# src/core/models.py
from datetime import date
from sqlalchemy import CheckConstraint, ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from src.core.db import db

class Servicio(db.Model):
    __tablename__ = "servicios"

    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(100), nullable=False)
```

`Mapped[str]` describe el atributo; no valida entradas de un formulario.

Modelo: una solicitud

```
# Continúa src/core/models.py; usa los imports anteriores.
class Solicitud(db.Model):
    __tablename__ = "solicitudes"
    __table_args__ = (
        CheckConstraint("cantidad > 0", name="ck_cantidad_positiva"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    servicio_id: Mapped[int] = mapped_column(
        ForeignKey("servicios.id"), nullable=False
    )
    fecha: Mapped[date] = mapped_column(nullable=False)
    cantidad: Mapped[int] = mapped_column(nullable=False)
```

La cantidad es **pedida**: no representa autorización ni entrega.

Leer el modelo como un contrato

- `primary_key` : identifica una solicitud.
- `ForeignKey` : referencia un servicio existente.
- `nullable=False` : impide guardar `NULL` .
- `CheckConstraint` : exige una cantidad positiva en la base.

La aplicación también valida y explica errores al usuario.

Pregunta: ¿alcanza con `min="1"` en el input HTML?

Declarar clases no crea tablas

Consola de Flask:

```
poetry run flask --app src.web:create_app shell
```

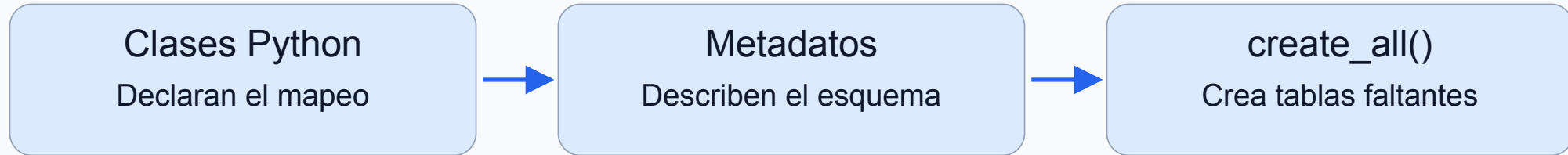
Dentro de esa consola, que tiene contexto de aplicación:

```
from src.core.db import db
from src.core.models import Servicio, Solicitud
from sqlalchemy import select
from datetime import date

db.create_all()
```

Los fragmentos siguientes reutilizan estos imports y este contexto.

Crear no es migrar



Las tablas existentes no se modifican: necesitan migraciones.

Cambiar una columna en Python **no modifica una tabla existente** con `create_all()`.

- Una migración describe cómo evolucionar el esquema.
- Debemos revisar su impacto sobre los datos existentes.
- Borrar y recrear tablas no es una estrategia para conservar datos.

Una base vacía necesita un esquema; una base existente necesita una evolución

Dos significados de «sesión»

Sesión web	Sesión del ORM
Estado entre requests de un cliente	Seguimiento de objetos y operaciones con la base
Asociada al mecanismo de sesión de Flask	Disponible aquí como <code>db.session</code>
No confirma escrituras SQL	Participa en la unidad de trabajo

Flask-SQLAlchemy gestiona su ámbito y limpieza al finalizar el contexto.

Limpiar la sesión no significa confirmar los cambios.

Crear un objeto no es guardarlo

```
servicio = Servicio(nombre="Pediatría")

# El objeto existe en Python.
db.session.add(servicio)

# La sesión conoce un objeto pendiente de inserción.
db.session.commit()
```

`Servicio(...)` crea el objeto; `add` lo incorpora a la sesión.

`commit` confirma la transacción si no se produce un error.

Consultar: construir y ejecutar

```
consulta = (  
    select(Solicitud)  
    .where(Solicitud.cantidad >= 20)  
    .order_by(Solicitud.fecha, Solicitud.id)  
    .limit(10)  
)  
  
solicitudes = db.session.scalars(consulta).all()
```

`select(...)` construye la expresión. La sesión la ejecuta.

`scalars()` entrega objetos `Solicitud`; `all()` reúne el resultado.

El filtro debe llegar a la base

```
# Evitar traer toda la tabla para filtrar en Python.  
todas = db.session.scalars(select(Solicitud)).all()  
resultado = [s for s in todas if s.servicio_id == servicio_id]
```

```
# Pedir sólo lo necesario.  
consulta = select(Solicitud).where(  
    Solicitud.servicio_id == servicio_id  
)  
.order_by(Solicitud.id).limit(50)  
resultado = db.session.scalars(consulta).all()
```

Para listados grandes necesitamos paginación, no un `all()` sin límites.

Buscar por identidad y contemplar la ausencia

```
solicitud = db.session.get(Solicitud, 10)

if solicitud is None:
    print("No existe la solicitud")
else:
    print(solicitud.cantidad)
```

El identificador puede no existir. No debemos inventar un resultado exitoso.

El controlador decide cómo traducir esa ausencia a HTTP.

Checkpoint: anticipar el resultado

```
consulta = select(Solicitud).where(Solicitud.cantidad > 100)
```

¿Qué sabemos al llegar a esta línea?

- A. Ya recibimos todas las solicitudes mayores a 100.
- B. Construimos una consulta, todavía no la ejecutamos.
- C. Actualizamos la cantidad de todas las solicitudes.

Relaciones: claves y objetos

Atributos de relación en `models.py` :

```
# En Servicio:
solicitudes: Mapped[list["Solicitud"]] = relationship(
    back_populates="servicio"
)

# En Solicitud:
servicio: Mapped["Servicio"] = relationship(
    back_populates="solicitudes"
)
```

`ForeignKey` define la referencia en la base.

`relationship` permite navegar objetos; no crea otra columna.

Crear usando la relación

```
servicio = db.session.get(Servicio, 1)
if servicio is None:
    raise LookupError("Servicio inexistente")

solicitud = Solicitud(
    servicio=servicio,
    fecha=date(2026, 9, 17),
    cantidad=20,
)
db.session.add(solicitud)
db.session.commit()

print(solicitud.servicio.nombre)
```

Asignamos un objeto. El ORM coordina la clave foránea al persistir.

Un JOIN sigue siendo un JOIN

```
consulta = (  
    select(Solicitud.id, Servicio.nombre, Solicitud.cantidad)  
    .join(Solicitud.servicio)  
    .where(Solicitud.cantidad >= 20)  
    .order_by(Solicitud.id)  
    .limit(20)  
)  
  
for identificador, nombre, cantidad in db.session.execute(consulta):  
    print(identificador, nombre, cantidad)
```

Seleccionamos varias columnas: recibimos filas, no sólo objetos `Solicitud`.
No usar `scalars()` si necesitamos todas esas columnas.

Actualizar un objeto recuperado

```
solicitud = db.session.get(Solicitud, 10)
if solicitud is None:
    raise ValueError("Solicitud inexistente")

solicitud.cantidad = 25
db.session.commit()
```

El ORM sigue los cambios del objeto dentro de la sesión.

Poder ejecutar el UPDATE no demuestra que el usuario esté autorizado.

Eliminar también exige una decisión

```
solicitud = db.session.get(Solicitud, 10)
if solicitud is None:
    raise ValueError("Solicitud inexistente")

db.session.delete(solicitud)
db.session.commit()
```

Eliminar una fila puede hacer perder información del dominio.

¿En el dominio corresponde eliminar, cancelar o conservar un historial?

No agregar borrados en cascada sin comprender qué registros afectarán.

add, flush, commit, rollback

Operación	Idea central
<code>add(objeto)</code>	Incorporar un objeto a la sesión
<code>flush()</code>	Enviar cambios pendientes dentro de la transacción
<code>commit()</code>	Hacer flush y confirmar la transacción
<code>rollback()</code>	Revertir la transacción activa

Que un INSERT se haya ejecutado no significa que esté confirmado.

Una consulta ORM puede provocar un flush automático antes de ejecutarse.

Un identificador no demuestra persistencia

```
temporal = Servicio(nombre="Servicio temporal")
db.session.add(temporal)
db.session.flush()

identificador = temporal.id
print(identificador)
db.session.rollback()

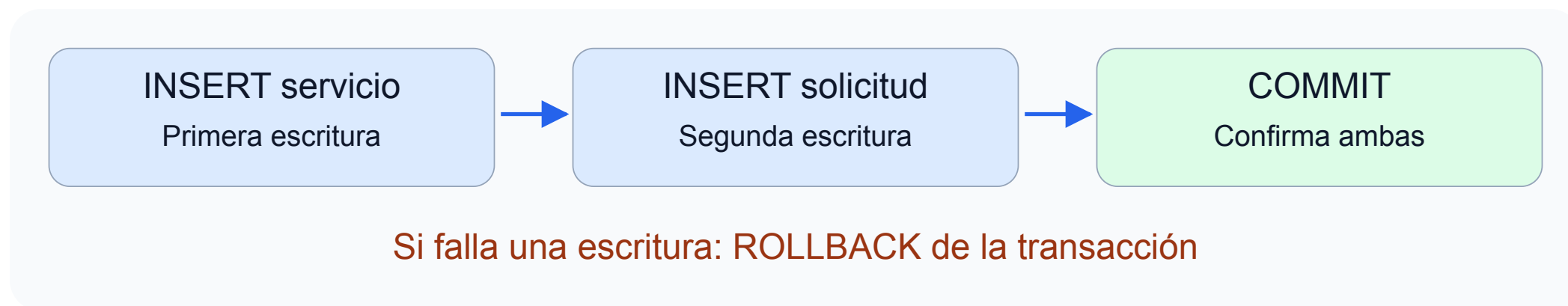
print(db.session.get(Servicio, identificador))  # None
```

El rollback afecta a toda la transacción activa, no sólo a este objeto.

El identificador se asignó, pero la inserción se revirtió; puede quedar un hueco.

Una operación puede necesitar varias escrituras

Crear un servicio y su primera solicitud puede ser **una sola operación**.



Si la solicitud falla, no queremos dejar creado el servicio a medias.
La frontera transaccional depende de la operación, no de cada tabla.

Una sola confirmación al final

```
# Unidad de trabajo independiente, sin cambios ajenos pendientes.  
try:  
    nuevo = Servicio(nombre="Consultorios")  
    primera = Solicitud(  
        servicio=nuevo, fecha=date(2026, 9, 17), cantidad=10  
    )  
    db.session.add_all([nuevo, primera])  
    db.session.commit()  
except Exception:  
    db.session.rollback()  
    raise
```

Aquí capturamos para revertir y **relanzamos**: no ocultamos el fallo.
No debe haber un commit intermedio que rompa la unidad de trabajo.

Provocar un error y recuperar

```
from sqlalchemy.exc import IntegrityError

try:
    invalida = Solicitud(
        servicio_id=1, fecha=date(2026, 9, 17), cantidad=-1
    )
    db.session.add(invalida)
    db.session.commit()
except IntegrityError:
    db.session.rollback()
    print("No se pudo guardar: se incumplió una restricción")
except Exception:
    db.session.rollback()
    raise
```

Tras fallar el flush, hay que hacer rollback antes de reutilizar la sesión.
Los demás errores se revierten y se propagan: no se informan como éxito.

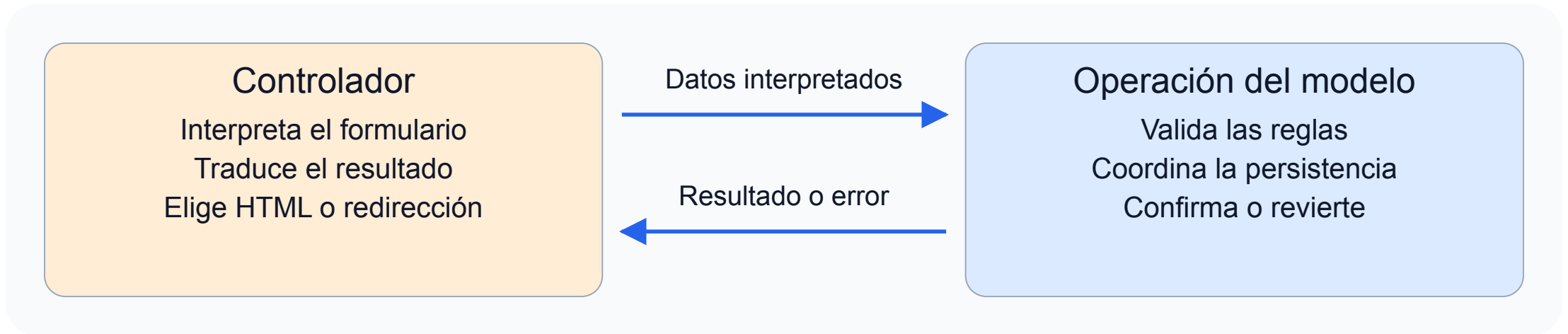
Diagnóstico: ¿qué evidencia buscamos?

1. Intentar guardar una cantidad negativa.
2. Identificar la restricción que rechazó la escritura.
3. Revertir la transacción y consultar de nuevo.
4. Comprobar que el dato inválido no quedó guardado.
5. Reintentar con datos válidos y verificar el resultado.

No alcanza con que desaparezca el traceback.

Una transacción no evita por sí sola los conflictos cuando dos operaciones modifican el mismo dato al mismo tiempo.

Volvemos a MVC



- El modelo no lee `request.form` ni genera HTML.
- El controlador no informa éxito antes de confirmar.
- La vista no decide cuándo guardar.

Un modelo de dominio es más que una clase que representa una tabla.

Una operación con frontera clara

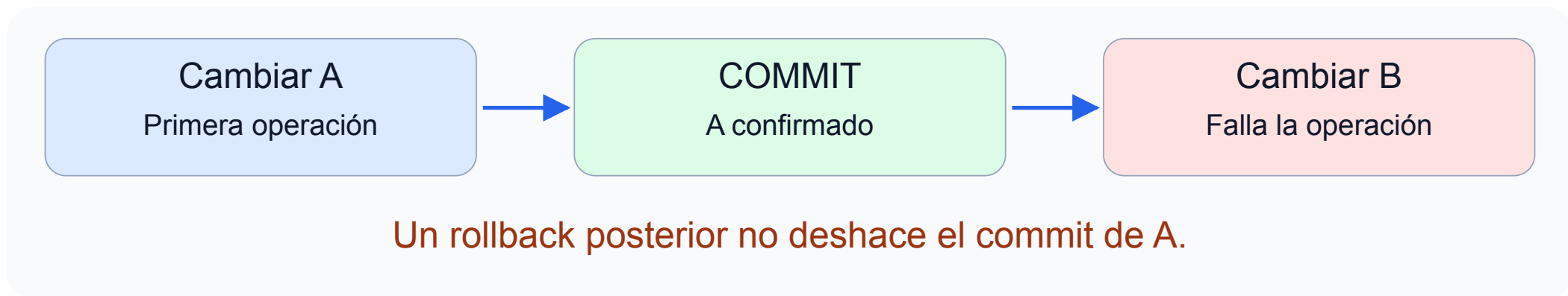
```
# src/core/solicitudes.py
from src.core.db import db
from src.core.models import Solicitud

def cambiar_cantidad(identificador, cantidad):
    if cantidad <= 0:
        raise ValueError("La cantidad debe ser positiva")
    try:
        solicitud = db.session.get(Solicitud, identificador)
        if solicitud is None:
            raise LookupError("Solicitud inexistente")
        solicitud.cantidad = cantidad
        db.session.commit()
    except Exception:
        db.session.rollback()
        raise
```

Contrato: recibe enteros validados y es dueña de esta unidad de trabajo.

¿Dónde no pondríamos ese commit?

Si una operación necesita **cambiar dos solicitudes juntas**, no puede llamar dos veces a una función que confirma cada cambio por separado.



Necesitamos una operación mayor que coordine ambas escrituras y confirme una vez.

La función anterior sirve para una modificación independiente, no como pieza transaccional universal.

Leer el SQL también es usar el ORM

En la factory, antes de `db.init_app(app)` :

```
app.config["SQLALCHEMY_ECHO"] = os.getenv("SQLALCHEMY_ECHO", "0") == "1"
```

Sin esa variable, el registro SQL queda desactivado. En desarrollo se puede activar explícitamente con `SQLALCHEMY_ECHO=1` .

Observar:

- qué SELECT se ejecuta y con qué filtro;
- cuándo aparecen INSERT y UPDATE;
- dónde ocurren COMMIT y ROLLBACK;
- si acceder a un atributo dispara otra consulta.

El registro puede incluir valores sensibles: protegerlo y usarlo con criterio.

Lectura colectiva: código propuesto por un agente

```
# src/core/solicitudes.py – propuesta para revisar
def registrar(datos):
    solicitud = Solicitud(**datos)
    db.session.add(solicitud)
    try:
        db.session.flush()
    except Exception:
        pass
    return "Solicitud registrada"
```

¿Qué afirmaciones no podemos sostener? ¿Qué pasa después de un error?

Revisar antes de aceptar

En el fragmento anterior:

- `flush` no reemplaza a `commit`.
- Se oculta el error y no se recupera la sesión.
- Se informa éxito sin haber confirmado el alta.
- Se aceptan atributos sin una selección explícita.

Además debemos revisar tipos, reglas y permisos.

El ORM no convierte datos externos en datos confiables.

¿Cuántas consultas ejecutamos?

```
servicios = db.session.scalars(  
    select(Servicio).order_by(Servicio.id).limit(10)  
)  
.all()  
  
for servicio in servicios:  
    print(servicio.nombre, len(servicio.solicitudes))
```

Con relaciones todavía no cargadas, puede haber una consulta inicial más otra por cada servicio: patrón **N + 1**.

Diez líneas de Python no equivalen a diez operaciones baratas.

Cargar relaciones deliberadamente

```
from sqlalchemy.orm import selectinload

consulta = (
    select(Servicio)
    .options(selectinload(Servicio.solicitudes))
    .order_by(Servicio.id)
    .limit(10)
)
servicios = db.session.scalars(consulta).all()
```

Para este conjunto pequeño: una consulta de servicios y otra de solicitudes. Las relaciones ya cargadas en la sesión pueden evitar consultas adicionales.

No cargar colecciones enormes si sólo necesitamos un conteo.

Pedir el resumen, no todas las filas

```
from sqlalchemy import func

consulta = (
    select(Servicio.nombre, func.count(Solicitud.id))
    .outerjoin(Servicio.solicitudes)
    .group_by(Servicio.id, Servicio.nombre)
    .order_by(Servicio.nombre, Servicio.id)
    .limit(20)
)
for nombre, total in db.session.execute(consulta):
    print(nombre, total)
```

`outerjoin` incluye servicios sin solicitudes; `count(Solicitud.id)` da cero.
El resultado cuenta solicitudes, no suma cantidades pedidas.

El error aparece «en la consulta»

```
invalida = Solicitud(  
    servicio=servicio, fecha=date(2026, 9, 17), cantidad=-1  
)  
db.session.add(invalida)  
  
# Puede fallar aquí por el flush automático previo al SELECT.  
servicios = db.session.scalars(select(Servicio)).all()
```

La línea que muestra el traceback no siempre inició el problema.

Recuperar con `db.session.rollback()` y revisar los cambios pendientes.

Desactivar autoflush no hace válido el dato ni evita el flush del commit.

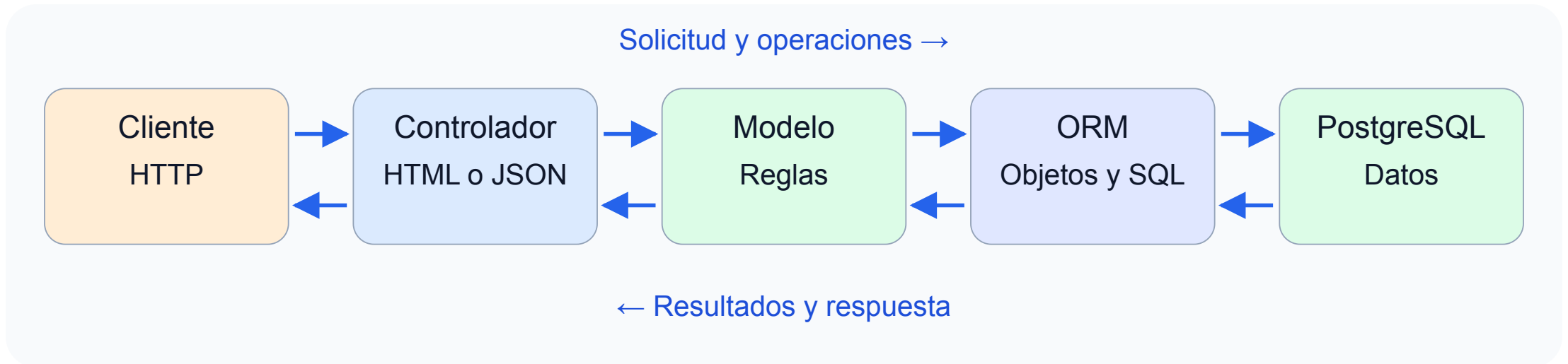
Qué debemos poder explicar

Antes de aceptar una operación con ORM:

1. ¿Qué filas busca o modifica?
2. ¿Qué restricciones y reglas deben cumplirse?
3. ¿Quién confirma y qué ocurre si falla?
4. ¿Qué SQL y cuántas consultas se ejecutan?
5. ¿Qué evidencia demuestra el resultado?

Estas preguntas valen tanto para código propio como para código generado.

Lo que sigue: un recurso persistente



Hoy el resultado puede terminar en un template.

En una API, construiremos una representación y una respuesta HTTP.

Cambiar HTML por JSON no cambia las garantías de persistencia.

Referencias para consultar

- Inicio de Flask-SQLAlchemy
- Modelos declarativos
- Consultas SELECT
- Relaciones
- Sesión y transacciones
- Carga de relaciones

Usamos el estilo de SQLAlchemy 2.x: `select` , `Mapped` y `mapped_column` .

Para cerrar

Trabajar con objetos no nos exime de entender la base.

El ORM reduce trabajo repetitivo.

La responsabilidad sobre los datos sigue siendo nuestra.